

SQL

SQL: Structured Query Language (Lenguaje de Consulta Estructurada), lenguaje para consultar, manipular y transformar datos de un base de datos relacional.

Permite crear tablas, insertar datos, actualizarlos, eliminarlos, hacer consultas, definir permisos, etc.

Lo implementan distintos sistemas de bases de datos como MySQL, Oracle, SQL Server, PostgreSQL, etc.

MySQL: sistema de gestión de bases de datos relacional (RDBMS) utiliza SQL para interactuar con los datos - estructura rígida con esquema.

Programa que corre en computadora/servidor, permite crear, organizar y consultar bases de datos usando SQL.

Open source (gratis en su versión comunitaria).

NoSQL (no relacional) - esquema dinámico o inexistente:

No trabaja con tablas fijas, sino con documentos, colecciones, grafos o pares clave-valor.

Más flexible, usado en Big Data, aplicaciones web rápidas.

ej. MongoDB, Cassandra, Redis.

DBMS: Data Base Management System

Dato: valor puntual, sin contexto. Ej. 65

Información: datos con significado. Ej. "El usuario pesa 65 kg"

Tabla básica

- Conjunto de columnas (atributos) y filas (registros).
- Columnas: representan las propiedades de la entidad, número fijo de columnas.

Atributos

- camelCase o snake_case
- Cada tabla puede tener tantos atributos como necesite.

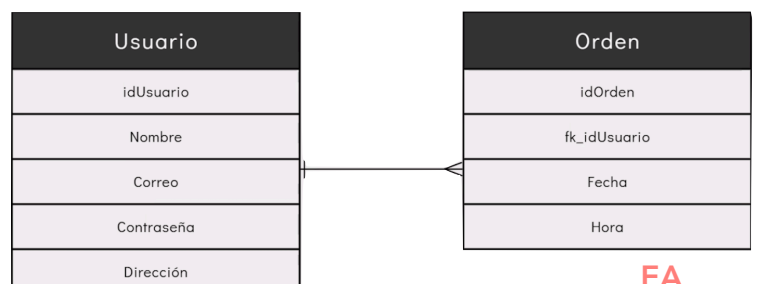
- Filas: representan un registro de datos. Cualquier número de filas de datos

Clave primaria (Primary Key, PK):

- Identificador único para cada registro de una tabla.
- Atributo esencial
- Ningún valor se repite ni puede estar vacío.
- Convención: nombreEntidad + ID → usuarioID, productoID.

Clave foránea (Foreign Key, FK)

- Enlace hacia la clave primaria de otra tabla
- Relaciona tablas.
- Convención: _ + nombreParametro + ID + _fk → usuarioID_fk.



EA

Tipos de relaciones entre tablas

- **1:1 (uno a uno - one to one):** Relaciona **un registro** de la tabla A **con uno solo** de la tabla B.
Ej. un coche - una matricula.
- **1:N (uno a muchos - one to many):** relación mas frecuente, **un registro** de tabla A se puede relacionar **con varios** de tabla B.
Ej. un usuario puede hacer multiples ordenes.
- **N:M (muchos a muchos - many to many):** **Muchos registros** de una tabla se relacionan **con muchos** de otra.
Se crea una tabla intermedia.
Ej. un producto tiene múltiples ventas, una venta tiene multiples productos.

No se puede guardar la relación directamente en una sola tabla porque repetiría datos y se rompería la integridad.

Tabla intermedia/pivote/puente: se crea para resolver relaciones **muchos a muchos entre entidades**.

Ej. order_has_product o
product_has_order.

id	idProducto_fk	idVenta_fk
1	1	3
2	1	2
3	2	1

Ej. estudiante_curso

Tablas principales:

Estudiantes → (id_estudiante, nombre)

Cursos → (id_curso, nombre)

Tabla pivote:

Estudiante_Curso → (id_estudiante, id_curso)

Si un estudiante se inscribe a 3 cursos, simplemente tendrá 3 filas en la tabla pivote (y no 3 registros duplicados en su propia tabla).

Si un curso tiene 20 estudiantes, la tabla pivote contendrá esas 20 relaciones.

Relaciones: evita registros duplicados dentro de la Base de Datos y garantiza la integridad de estos.



Abstracción de datos

Modelo de base de datos solo guarda lo esencial para representar la realidad.

Base de datos relacional: colección de tablas relacionadas. Sistema en el que la información se organiza en tablas que se relacionan entre sí por medio de llaves (claves).

Ej. una tabla Usuarios y otra Pedidos, relacionadas por el usuarioID.

Modelo entidad-relación (ER)

- **Entidad:** corresponde a una tabla (ej. Usuario, Producto).
- **Atributos:** columnas de la tabla.
- **Relaciones:** cómo se conectan las entidades (líneas entre tablas).
- **Cardinalidad:** 1:1, 1:N, N:M.

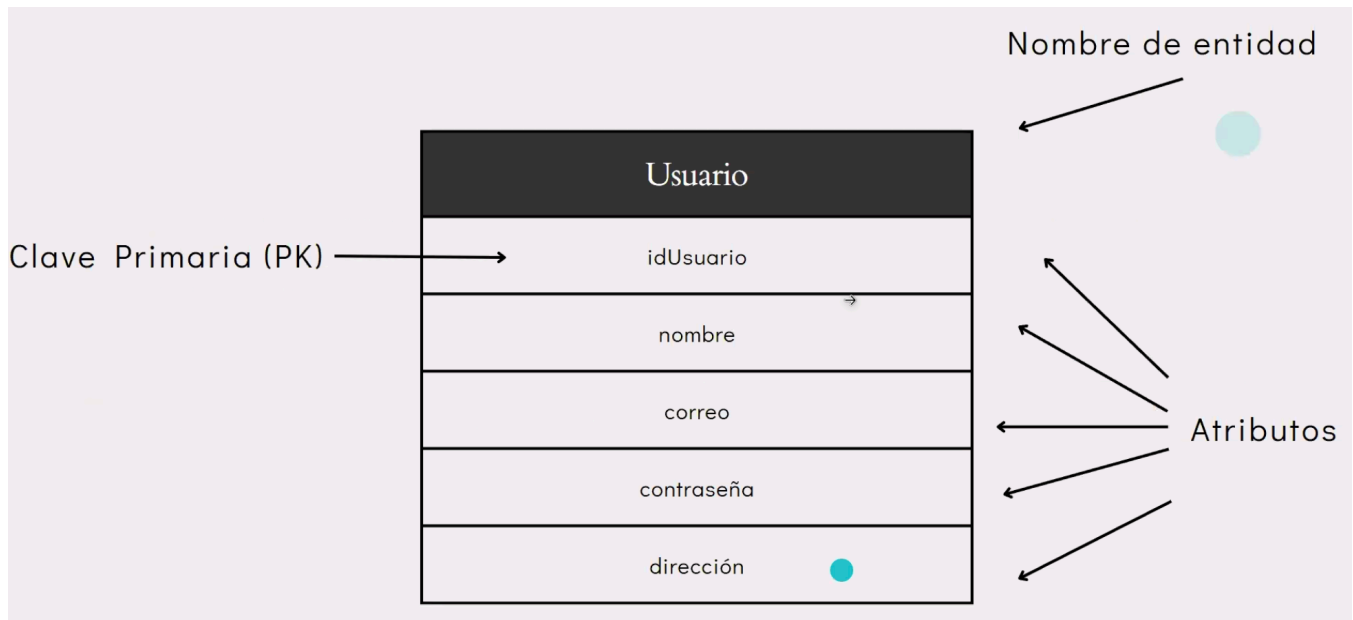
PK → identifica de forma única.

FK → conecta tablas.

Entidad → tabla.

Relación → cómo se vinculan las entidades (1:1, 1:N, N:M).

Modelo ER → representación gráfica de todo lo anterior.



Datos

Categoría	Tipo dato	Uso/notas	Ejemplo
Numéricos enteros	TINYINT	Enteros pequeños	127 / -128
	SMALLINT	Enteros medianos	32,767 / -32,768
	INT o INTEGER	Enteros grandes (IDs)	2,147,483,647
	BIGINT	Enteros muy grandes	valores enormes
Numéricos decimales	DECIMAL(10,2)	Exacto, ideal para dinero	\$12345.67
	FLOAT	Decimales aproximados (cálculos rápidos)	12.3456
	DOUBLE / REAL	Mayor precisión que FLOAT	1234567.8910
Texto (cadenas) alfanuméricos	CHAR(n)	Longitud fija (rellena con espacios) de 1 a 255 caracteres	"ABC"
	VARCHAR(n)	Longitud variable (hasta n) mas optimo en almacenamiento en memoria	"Hola mundo"
	TEXT	Artículos, descripciones	Texto largo
Fechas y tiempos	DATE	Solo fecha día, mes y año	2025-08-23 YYYY-MM-DD
	TIME	Solo hora	14:35:59 HH:MM:SS
	DATETIME	Fecha + hora	2025-08-23 14:35:59
	TIMESTAMP	Fecha + hora con zona horaria	2025-08-23 20:35:59 UTC
	YEAR	Solo año	2025
Lógicos	BIT	se aplica logica booleana	En MySQL se guarda como TINYINT(1) , 0 = false y 1 distinto a 0 = true
	BOOLEAN	TRUE / FALSE	
Especiales (Binarios; no numéricos, no texto)	BLOB (Binary Large Object)	Datos binarios grandes hasta 65,535 bytes (64 KB)	(imagen, pdf)
	ENUM	Solo acepta uno de los valores predefinidos	('pendiente', 'pagado')
	SET	Conjunto de valores Puede almacenar varias opciones	('rojo', 'verde', 'azul')

SQL

<https://dev.mysql.com/doc/refman/8.4/en/data-types.html>

Datos numéricos

INTEGER: Valor entero

NUMERIC (n,m): números hasta 18 dígitos (con decimales)

n= total de dígitos admitidos

m= numero de posiciones decimales

DECIMAL (n,m): = numeric

Null: Representa la ausencia de valor (no es lo mismo que 0 o "").

Al comparar con **NULL**, no funciona el =, se usa **IS NULL** o **IS NOT NULL**.

`SELECT * FROM Usuario WHERE correo IS NULL;`

Devuelve los usuarios sin correo registrado.

Bytes y Bits

Bit: unidad mínima de información (0 o 1).

Byte: conjunto de **8 bits**.

Kilobyte (KB): 1024 bytes.

Megabyte (MB): 1024 KB.

Gigabyte (GB): 1024 MB.

ej.

Una letra en ASCII = **1 byte = 8 bits**.

Una imagen pequeña en PNG = puede ocupar varios **KB o MB**.

LENGTH() devuelve el número de bytes almacenados en una columna de texto/binaria.

Para arreglos (propiedad) y cadenas en programación (método) (Java, JS, etc.):

.length → cuenta número de elementos/caracteres.

.length() → método en algunas clases (ej. Java String).

En SQL no es propiedad, es función (**LENGTH()** o **CHAR_LENGTH()**).

SQL

```
CREATE TABLE Producto (  
    productoID INT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    precio DECIMAL(10,2),  
    descripcion TEXT,  
    fechaAlta DATETIME,  
    disponible BOOLEAN  
);
```

productoID → entero único.

nombre → texto corto.

precio → número con 2 decimales.

descripcion → texto largo.

fechaAlta → fecha y hora de alta.

disponible → true/false.

DML: Data Manipulation Language, manipulación de datos

Comandos:

- Select
- Update
- Insert
- Delete from

DDL: Data Definition Language, define base de datos

Comandos:

- Create table
- Create index
- Create view

DCL: Data Control Language, gestionar permisos y usuarios

Comandos:

- Grant otorga permisos (ej. GRANT SELECT ON tabla TO 'usuario';).
- Revoke (ej. REVOKE SELECT ON tabla FROM 'usuario';)

TCL: Transaction Control Language, registro de modificaciones (grupo de operaciones que se ejecutan juntas)

Comandos:

- Begin/ Start transaction: inicia una transacción.
- Commit: guarda los cambios de forma permanente.
- Rollback: revierte cambios al último punto seguro.
- Savepoint: crea un "punto de guardado" dentro de una transacción para poder volver ahí.
- Set transaction: define características de la transacción (ej. nivel de aislamiento).

Select from: Selecciona columnas específicas

```
SELECT*  
FROM tabla;
```

Select distinct ... from: Elimina duplicados, devolviendo solo valores únicos

Select: instrucción para consultar datos.

From: indica de qué tabla traer los datos.

```
SELECT DISTINCT ciudad  
FROM clientes;
```

Devuelve lista de ciudades únicas en la tabla clientes.

Order-by: Ordena los resultados

ASC → ascendente (por defecto).

DESC → descendente.

```
SELECT nombre, edad FROM Usuario ORDER BY edad DESC;
```

*texto en orden alfabético

Lista usuarios de mayor a menor edad.

Se utilizan clausulas como:

LIMIT → especifica un número limitado de filar que devolverá una consulta.

OFFSET → opcional. Determina la primera fila devuelta por limit.

Group by: Agrupa filas que tienen valores iguales en ciertas columnas. Se usa junto con funciones de agregación.

```
SELECT ciudad, COUNT(*) AS total
```

```
FROM Usuario
```

```
GROUP BY ciudad;
```

Cuenta cuántos usuarios hay por ciudad.

Alias: nombre temporal para una columna o tabla, útil para hacer más legible una consulta.

```
SELECT nombre AS cliente, edad AS años FROM Usuario;
```

AS cliente y AS años son alias.

*uso de backticks (`) para evitar conflicto con palabras reservadas.

SQL

Where: Filtra registros en una consulta, si cumple la condición la consulta devuelve los valores que se relacionan con la condición que se especifique en la cláusula WHERE.

Se puede usar con las instrucciones UPDATE y DELETE además de la instrucción WHERE

```
SELECT * FROM Producto WHERE precio > 500;
```

Devuelve productos que cuestan más de 500.
Se combina con operadores (AND, OR, LIKE, etc.).

Única cláusula de comparación - **condición simple**

Múltiples cláusulas - operadores AND u OR - **condición compuesta**

Having: Es como un WHERE (incluir condiciones con alguna función SQL), aplicado después de agrupar con GROUP BY.

```
SELECT ciudad, COUNT(*) AS total
FROM Usuario
GROUP BY ciudad
HAVING COUNT(*) > 10;
```

Solo muestra las ciudades con más de 10 usuarios.

SQL

Operadores

Aritméticos:

+ suma, - resta, * multiplicación, / división, % módulo.

```
SELECT precio * 1.16 AS precioConIVA FROM Producto;
```

Lógicos:

AND → ambas condiciones deben cumplirse.

OR → una u otra condición.

NOT → niega condición.

```
SELECT * FROM Usuario WHERE edad > 18 AND ciudad = 'CDMX';
```

De texto:

LIKE → buscar patrones con % (cualquier cadena) y _ (un carácter).

```
SELECT * FROM Usuario WHERE nombre LIKE 'A%';
```

Todos los nombres que empiezan con A.

%texto → termina en "texto".

texto% → empieza con "texto".

%texto% → contiene "texto".

IN → registros que coinciden con una lista de valores.

```
SELECT * FROM Usuario WHERE ciudad IN ('CDMX','Monterrey');
```

BETWEEN → valores en un rango.

```
SELECT * FROM Producto WHERE precio BETWEEN 100 AND 500;
```

*texto en orden alfabético

*numérico en orden de menor a mayor

SQL

Insert into: Agregar registros a una tabla.

```
INSERT INTO Usuario (usuariolD, nombre, edad, correo)
VALUES (1, 'Ana', 25, 'ana@email.com');
```

*recordar el uso de ' ' en texto y fechas

SINTAXIS

1. Con todas las columnas en orden

```
INSERT INTO empleado VALUES (1, 'Erika', 'Hernandez', 30000);
```

Requiere respetar el **orden de las columnas** de la tabla.

2. **Especificando columnas** (más recomendado)

```
INSERT INTO empleado (id, nombre, apellido, salario)
VALUES (2, 'Rodrigo', 'Gomez', 28000);
```

3. **Insertando múltiples filas** en una sola sentencia

```
INSERT INTO empleado (id, nombre, apellido, salario)
VALUES
(3, 'Ana', 'Perez', 25000),
(4, 'Emi', 'Cervantes', 27000);
```

*No se puede insertar en medio y que se recorran los datos como en una lista.

***Duplicate entry for PRIMARY KEY**, Para **reemplazar** se puede usar: INSERT OR REPLACE (SQLite), REPLACE INTO (MySQL), O **mejor práctica** → **UPDATE**.

No hay error → si se elimina antes el registro, entonces se puede reutilizar el ID (atributo) manualmente.

Si no borras antes → usa UPDATE o REPLACE.

Si **ya borraste antes** → volver a insertar con ese mismo ID sin problema.

Update: Actualiza registros existentes.

```
UPDATE Usuario
SET correo = 'nuevo@email.com'
WHERE usuariolD = 1;
```

Cambia el correo del usuario con ID = 1.

⚠ si se omite WHERE, se actualizan todas las filas.

Delete: Elimina registros de una tabla.

```
DELETE FROM Usuario WHERE usuariolD = 5;
```

*importante el uso de WHERE condición; si no borra toda la tabla

Create table: Crear una nueva tabla dentro de la BD.

```
CREATE TABLE empleado (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(50) NOT NULL,  
    apellido VARCHAR(50),  
    salario DECIMAL(10,2),  
    departamento VARCHAR(50) DEFAULT 'Admin'  
);
```

ALTER TABLE: Agrega, elimina o modifica columnas de una tabla existente.

Renombrar columna

```
ALTER TABLE empleado  
RENAME COLUMN apellido TO last_name;
```

Renombrar columna

```
ALTER TABLE empleado  
RENAME TO empleados;
```

Añadir columna

```
ALTER TABLE nombre tabla  
ADD nombreColumna tipoDato;
```

Borrar columna

```
ALTER TABLE nombre tabla  
DROP COLUMN nombreColumna;
```

Modificar tipo de dato de una columna

```
ALTER TABLE nombre tabla  
MODIFY nombreColumna tipoDato;
```

DROP – Borrado de objetos

Borrar tabla

```
DROP TABLE nombreTabla;
```

Borrar vistas

```
DROP VIEW nombre_Vista;
```

Borrar índice

```
DROP INDEX nombre_índice ON tabla;
```

Borrar Base de Datos

```
DROP DATABASE nombre_db;
```

Si solo se quiere borrar datos pero dejar la estructura

```
TRUNCATE TABLE empleado;
```

SQL

***modo seguro:** Evita actualizaciones o borrados sin usar **WHERE** con clave primaria o índice.

Preferences → SQL Editor → Safe Updates (rejects UPDATEs and DELETEs without key in WHERE clause).

Funciones de agregación

Sirven para resumir información

```
SELECT MIN(nombreColumna) FROM nombreTabla;
```

MIN() → mínimo.

MAX() → máximo.

COUNT() → contar registros, numero de filas que cumplen la condición, valores null se ignoran. COUNT * devuelve numero de filas de una tabla.

AVG() → promedio.

SUM() → suma de valores.

SQL

Join: combina filas de dos o mas tablas según una columna relacionada entre ellas, basándose en una condición que las relacione, normalmente una clave primaria (PK) y una clave foránea (FK).

```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

- A. Select son las columnas que se van a visualizar
- B. La tabla que está a la izquierda/derecha de la palabra JOIN/LEFT es la que se conserva sí o sí.

Todas las tablas que se vayan agregando después, se van uniendo a esa base.

Si hay coincidencias → aparecen los datos.

Si no hay coincidencias → se rellena con NULL.

*Cuando se van encadenando varios JOIN, cada nueva tabla se compara con el resultado acumulado de las anteriores, no directamente con la primera.

Cuando se van encadenando varios JOIN, cada nueva tabla se compara con el resultado acumulado de las anteriores, no directamente con la primera.

Ej. + de 2 tablas

```
SELECT *
FROM tabla1 t1
LEFT JOIN tabla2 t2 ON t1.id = t2.t1_id
LEFT JOIN tabla3 t3 ON t2.id = t3.t2_id;
```

Une `tabla1` con `tabla2`.

Obtiene un resultado intermedio (con coincidencias o NULL).

Luego ese resultado intermedio se une con `tabla3`.

La condición (`t2.id = t3.t2_id`) usa datos de la **unión previa**.

Es decir, `tabla3` no se compara directamente con `tabla1`, sino con el resultado ya unido de `tabla1 + tabla2`.

- C. **INNER, LEFT, RIGHT** → buscan **coincidencias** (ej. usuario con compras).
- CROSS JOIN** → no necesita condición (ON) porque no busca coincidencias. Simplemente combina **todo con todo**. (Si la tabla A tiene **m** filas y la tabla B tiene **n** filas, el CROSS JOIN genera **m × n** filas).

SQL

Join	Uso/notas	Ejemplo
Inner join o Join	Solo las filas que tienen coincidencia en ambas tablas. Es el más común.	SELECT usuarios.nombre, compras.producto FROM usuarios INNER JOIN compras ON usuarios.id = compras.usuario_id;
Left join o Left outer join	Todos los de la izquierda + coincidencias , aunque no tengan coincidencia en la derecha. Si no hay coincidencia, los valores de la derecha aparecen como NULL .	SELECT usuarios.nombre, compras.producto FROM usuarios LEFT JOIN compras ON usuarios.id = compras.usuario_id;
Right join o Right outer join	Todos los de la derecha + coincidencias , aunque no tengan coincidencia en la izquierda. Opuesto al LEFT JOIN .	SELECT usuarios.nombre, compras.producto FROM usuarios RIGHT JOIN compras ON usuarios.id = compras.usuario_id;
Full join	Todos de ambos lados (coincidan o no) (No existe directamente en MySQL)	-
SQL union	(FULL OUTER JOIN) Une resultados de LEFT JOIN y RIGHT JOIN .	SELECT usuarios.nombre, compras.producto FROM usuarios LEFT JOIN compras ON usuarios.id = compras.usuario_id UNION SELECT usuarios.nombre, compras.producto FROM usuarios RIGHT JOIN compras ON usuarios.id = compras.usuario_id;
Cross	Producto cartesiano (todas las combinaciones posibles de filas entre las tablas). No necesita condición ON .	SELECT usuarios.nombre, productos.nombre FROM usuarios CROSS JOIN productos; *Si hay 5 usuarios y 3 productos = 15 combinaciones .

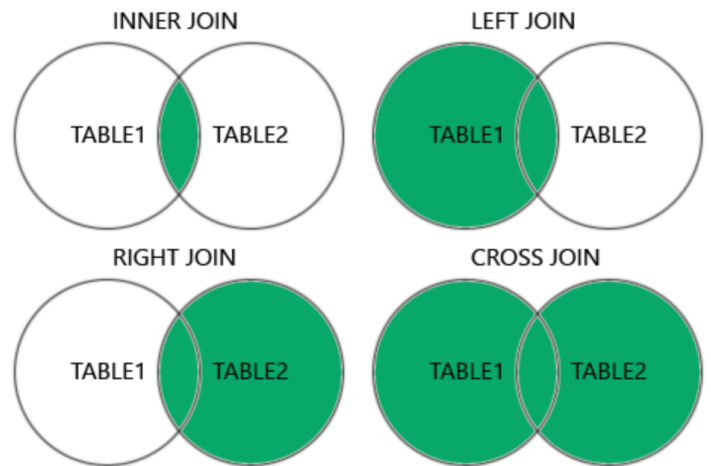
SQL

INNER JOIN → solo la intersección (coincidencias).

LEFT JOIN → toda la tabla izquierda + coincidencias.

RIGHT JOIN → toda la tabla derecha + coincidencias.

FULL OUTER JOIN → todo de ambas tablas, coincidan o no.



Formas de escribir un Join

1 trae toda la info de las tablas

2 trae las columnas que le pedi con select

3 como la 1 especifica traer toda la info de ambas tablas

```
SELECT *
FROM employee
INNER JOIN bonus
ON employee.employee_id = bonus.employee_id_fk;

SELECT employee.employee_id, employee.first_name, employee.last_name, employee.department, bonus.bonus_amount
FROM employee
INNER JOIN bonus
ON employee.employee_id = bonus.employee_id_fk;

SELECT employee.*, bonus.*
FROM employee
INNER JOIN bonus
ON employee.employee_id = bonus.employee_id_fk;
```

SQL

Indices: Mejoran la velocidad de las consultas en tablas. Crea un índice en la columna para buscar más rápido.

```
CREATE INDEX idx_apellido ON empleado(apellido);
```

apellido es la columna, pero idx_apellido es el **nombre del índice**.

Vistas: Consultas guardadas como tablas virtuales.

No almacenan datos, solo la consulta.

Da seguridad y comodidad.

Útiles para simplificar queries complejas.

```
CREATE VIEW nombreVista AS
SELECT nombreColumna, (S)
FROM nombre.tabla
WHERE condición;
```

Ahora se puede hacer:

```
SELECT * FROM UsuariosActivos;
```

Alias → nombres temporales.

Constraints → reglas de integridad.

Delete → borrar registros.

Distinct → quitar duplicados.

Funciones → cálculos sobre datos (MIN, MAX, COUNT, AVG, SUM).

Group By → agrupar resultados.

Having → filtrar grupos.

Indices → optimización de consultas.

Insert Into → insertar registros

Null → valor desconocido.

Operadores → aritméticos, lógicos y de texto (LIKE, IN, BETWEEN).

Order By → ordenar resultados.

Update → modificar registros.

Vistas → consultas almacenadas como tablas virtuales.

Where → filtrar registros.

Recursos

Mapeo visual: <https://www.drawio.com/>

Diagramas ER (Entidad-Relación).

Visualizar relaciones entre tablas (PK y FK).

Organizar consultas SQL en diagramas.

Draw.io → herramienta para diagramar bases de datos y relaciones.

Flujo de trabajo en workbench MySQL

Constraints (Restricciones)

Reglas que aseguran la integridad de los datos en una tabla.

Limitan el tipo de dato que puede recibir una columna de una tabla.

Se pueden definir cuando se crea una tabla (CREATE TABLE) o posteriormente con la sentencia ALTER TABLE.

PRIMARY KEY → valor único, no puede repetirse ni ser NULL, 1 por tabla.

FOREIGN KEY → referencia a otra tabla, una columna o varias que señalan la clave primaria solo pueden tener valores existentes en la clave primaria.

NOT NULL → no se permiten valores vacíos, siempre debe tener un valor.

UNIQUE → no se repite el valor, puede haber NULL, se puede aplicar varias veces.

BINARY → tipo de dato, guarda datos binarios puros (secuencias de bytes).
cadenas: CHAR(10).

UNSIGNED → solo valores positivos, aplica a tipos numéricos (INT, DECIMAL, etc.), quita los valores negativos y duplica el rango positivo.

ZEROFILL → rellena con ceros a la izquierda un número, según el ancho definido, automáticamente convierte la columna en UNSIGNED.

AUTO_INCREMENT → genera valores automáticos consecutivos, (numéricas, normalmente PRIMARY KEY)

GENERATED → su valor se calcula automáticamente en base a otra.

Virtual (VIRTUAL): no ocupa espacio, se calcula al vuelo.

Almacenada (STORED): se guarda en la tabla como si fuera una columna normal.

CHECK → validar condiciones, limitar rango de valores, se pueden definir varias (ej: edad > 18).

DEFAULT → proporciona un valor predeterminado a una columna.

Storage: ☐ Virtual ☐ Stored

☒ Primary Key ☒ Not Null ☐ Unique

☐ Binary ☐ Unsigned ☐ Zero Fill

☐ Auto Increment ☐ Generated

SQL

-- Comentario SQL

/*

Comentario de multiples líneas

*/

-- Muestra todas las bases de datos disponibles en mi servidor

SHOW DATABASES;

-- Elegir con que badabase trabajar

USE database;

-- Muestra tablas de la selección

SHOW TABLES;

Diagramas

FOREIGN KEY

Posterior a elegir la relación. Primero seleccionar la entidad donde se desea la foreign key.

Opciones de **ON UPDATE** y **ON DELETE**

Cuando se actualiza o elimina un registro en la tabla padre, se define qué pasa en la tabla hija:

CASCADE: los cambios se propagan.

Ej: si borras un cliente, también se borran sus pedidos.

SET NULL: pone la FK en NULL.

Ej: borras un cliente y en pedidos.cliente_id queda NULL.

SET DEFAULT: pone la FK en un valor por defecto.

RESTRICT / NO ACTION: evita la acción si hay registros relacionados.

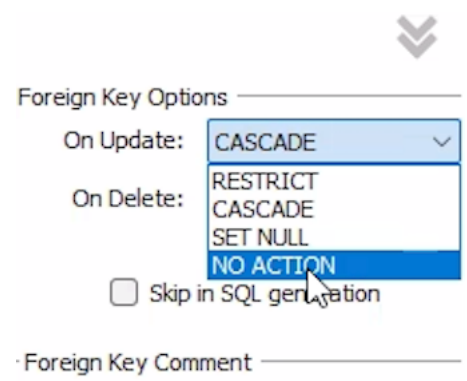
Ej: no puedes borrar un cliente que aún tenga pedidos.

Workbench en el Editor de Foreign Keys, al configurar las relaciones.

Skip in SQL Generation

Al crear tablas o relaciones, significa que ese elemento (columna, constraint, relación, índice) no será incluido en el script SQL final al exportar o forward-engineer el modelo.

Se usa si solo quieres definir algo para documentación o diseño, pero no ejecutarlo en la base real.



SQL

Abrir un script

- **Menú:** File → Open SQL Script...
- **Atajo:** Ctrl + O (Windows/Linux) o Cmd + O (Mac)
- Abre el explorador de archivos → seleccionar archivo .sql.
- Uso de ⚡ para ejecutar, uso de 🔄 en el panel derecho para reflejar la base de datos en esquemas.

Guardar un script

- **Menú:** File → Save Script (guarda con el mismo nombre).
- **Menú:** File → Save Script As... (guardar con otro nombre o ruta).
- **Atajo:** Ctrl + S o Cmd + S.

Guardar/Exportar archivos SQL

Exportar base de datos/tablas

- **Menú:** Server → Data Export...
- Eliges la base de datos y tablas.
- Seleccionas destino (archivo .sql en tu PC).
- Clic en **Start Export**.

Exportar resultado de una consulta

- Corre tu SELECT.
- En la tabla de resultados → clic derecho → Export Resultset...
- Elegir formato: .sql, .csv, .json, .xml, .html.

Conexiones

- **Menú:** Database → Connect to Database...
- Shortcut: Ctrl + U / Cmd + U.
Sirve para abrir tu conexión a MySQL (host, puerto, usuario, contraseña).

Importar datos

- **Menú:** Server → Data Import...
- Puedes importar desde:
 - Archivos .sql (dump).
 - Archivos .csv → opción *Import from Self-Contained File* → seleccionas tabla de destino.

Modelado y diagramas

- **Menú:** File → New Model
- Para abrir un EER (diagrama entidad-relación).
- **Forward Engineering:** Database → Forward Engineer (genera base desde el modelo).

SQL

- **Reverse Engineering:** Database → Reverse Engineer (genera modelo desde la BD real).

Explorar esquemas y tablas

- Panel izquierdo: pestaña **SCHEMAS**.
- Clic derecho en una tabla:
 - Select Rows - Limit 1000 → vista rápida de registros.
 - Copy SQL to Clipboard → genera query de esa tabla automáticamente.

Snippets y utilidades

- Panel derecho: **Snippets** (guardar consultas frecuentes).
- Menú Edit → Format → Beautify Query (Ctrl + B) → autoformatea tu SQL.